SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA37 **COURSE NAME: Software Testing**

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow. *(BL3)*

Assessment Tool Weightage: 30%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

Reverse Engineering Task

Code of Conduct:

*I **S.Dharmika/192421094(**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S.Dharmika

Annexure A

Reverse Engineering Task

Q1. Reverse Engineering of a Black-Box Web Application Using Selenium

Task Title: Reverse Engineering and Test Case Design for a Web-Based Login System

Tools: Selenium WebDriver, Chrome DevTools, Postman (optional), Excel or Google Sheets

A university uses an existing student portal for login, attendance checking, internal mark viewing, and course registration. The original requirements document and design specifications are unavailable. Students must reverse engineer the application's observable behavior and develop a systematic test suite without assuming access to its internal implementation.

The application contains the following functions:
- Username and password authentication.
- Login validation and error-message display.
- Session creation after successful login.
- Logout and session termination.
- Protection against unauthorized access to internal pages.

Practical tasks

a) Application reconnaissance and reverse engineering

1. Explore the application's available pages, forms, buttons, links, input fields, and navigation paths.
2. Use Chrome DevTools to inspect HTML elements, attributes, network requests, and browser-console errors.
3. Prepare a functional model showing the relationships between the login page, dashboard, attendance page, marks page, and logout function.
4. Derive a preliminary set of functional requirements from the observable behavior.
5. Clearly distinguish verified observations from assumptions about the hidden implementation.

b) Black-box test design

1. Identify valid and invalid equivalence classes for usernames and passwords.
2. Apply Boundary Value Analysis to any documented input-length restrictions.
3. Design positive, negative, and error-handling test cases.
4. Use random testing to select additional input combinations and identify unexpected behavior.
5. Create a test-case table containing Test Case ID, Preconditions, Input, Steps, Expected Result, Actual Result, Status, and Defect ID.

c) Automation using Selenium

1. Automate valid login, invalid login, empty-field validation, logout, and unauthorized-page access tests.
2. Use suitable element locators and explicit waits.
3. Capture screenshots when tests fail.
4. Store the test execution results in a structured report.

d) State and session testing

Construct a state transition diagram for Logged Out, Login in Progress, Logged In, and Session Expired states. Test valid and invalid transitions, including browser navigation after logout and session timeout where supported.

e) Test adequacy evaluation

Calculate the percentage of planned test cases executed and passed. Identify untested functions, prioritize risks, and explain whether the available evidence is sufficient to conclude that the application is ready for release.

Expected deliverables: Functional model, test scenarios, test-case spreadsheet, Selenium scripts, execution screenshots, defect report, state diagram, and test adequacy report.

Q2. Reverse Engineering of an Existing Java Application Using JaCoCo

Task Title: White-Box Reverse Engineering and Code Coverage Analysis of a Student Result Processing Application

Tools: Java, IntelliJ IDEA or Eclipse, JUnit, JaCoCo, Maven

A university has an existing Java application that calculates student totals, averages, grades, and pass/fail results. The application source code is available, but its original design document and test cases are missing. Several incorrect results have been reported, particularly for boundary marks and invalid inputs.

Students must reconstruct the program's logic, design tests, and evaluate the adequacy of the test suite using code coverage tools.

Sample program

```
public class ResultCalculator {

   public static String calculateGrade(int mark) {

      if (mark < 0 || mark > 100) {
         return "Invalid";
      }

      if (mark >= 90) {
         return "A";
      } else if (mark >= 75) {
         return "B";
      } else if (mark >= 50) {
         return "C";
      } else {
         return "Fail";
      }
   }
}
```

Practical tasks

a) Reverse engineering and logic reconstruction

1. Analyze the program and identify its inputs, processing logic, conditions, and possible outputs.
2. Prepare a flowchart and Control Flow Graph (CFG).
3. Identify decision nodes, branches, independent execution paths, and possible exceptional inputs.
4. Derive the implied grading rules and identify assumptions that require confirmation.

b) Black-box test design

1. Apply Equivalence Class Partitioning for marks below 0, between 0 and 100, and above 100.
2. Apply Boundary Value Analysis to the grade thresholds 50, 75, and 90.
3. Prepare test cases for values such as −1, 0, 49, 50, 74, 75, 89, 90, 100, and 101.
4. Use random testing to generate additional valid and invalid mark values.
5. Compare expected and actual outputs and document any discrepancies.

c) White-box test design

1. Identify the independent paths in the program.
2. Calculate cyclomatic complexity using \(V(G)=E-N+2P\).
3. Develop JUnit test cases to exercise the important true and false branches.

4. Compare statement coverage, branch coverage, and condition coverage.
5. Explain why executing every statement does not necessarily test every decision outcome.

d) Tool-based coverage analysis

1. Configure JaCoCo in a Maven-based Java project.
2. Execute the JUnit test suite and generate the HTML coverage report.
3. Inspect instruction, line, branch, method, and complexity information available in the report.
4. Identify uncovered lines and branches.
5. Add tests to improve coverage and regenerate the report.

e) Test adequacy evaluation

Compare the initial and improved test suites. Explain whether the coverage results establish that all grading requirements are correct. Identify additional checks needed for requirement correctness, invalid input handling, and untested combinations.

Expected deliverables: Reconstructed flowchart, CFG, complexity calculation, JUnit test scripts, JaCoCo HTML report screenshots, test results, defect log, and before-and-after coverage comparison.

Q3. Reverse Engineering and API Testing of a Banking Application

Task Title: Reverse Engineering of a Banking REST API and Validation of Transaction Rules

Tools: Postman, browser developer tools, Swagger/OpenAPI (if available), Excel

A bank provides a transaction application with APIs for retrieving account balances and initiating fund transfers. The API documentation is incomplete. Testers must infer the observable request-response behavior, reconstruct the transaction workflow, and test the application's business rules.

Use an authorized training API, a mock API, or a locally hosted sample application. Do not test a real banking system without written authorization.

Assume the following intended transaction rules:

- The transfer amount must be greater than zero.
- The transfer amount must not exceed the available balance.
- The transfer must not exceed the daily transaction limit.
- An unauthenticated request must not complete a transfer.
- A failed transaction must not incorrectly reduce the balance.

Practical tasks

a) API reverse engineering

1. Identify available endpoints, HTTP methods, request parameters, response codes, and response structures.
2. Use Postman to submit valid and invalid requests to the authorized test environment.
3. Record the observed request-response pairs and infer the API's functional behavior.
4. Prepare an API interaction diagram showing authentication, balance retrieval, transfer submission, and transaction response.
5. Document any behavior that cannot be confirmed from the available interface.

b) Black-box test design

1. Apply Equivalence Class Partitioning to transaction amounts.
2. Apply Boundary Value Analysis to the minimum permitted amount, available balance, and daily limit.
3. Create decision-table tests covering authentication, balance sufficiency, and limit validation.
4. Apply random testing to valid numeric values and malformed request parameters.
5. Design negative tests for missing fields, incorrect data types, invalid account identifiers, and unsupported HTTP methods.

c) State transition and transaction testing

Model the transaction states as Pending, Successful, Failed, and Cancelled where supported by the application. Test valid transitions and investigate whether repeated requests can create duplicate transfers.

d) White-box and control-flow analysis

If source code is supplied, construct a CFG for the transaction-validation logic, identify decision paths, and calculate cyclomatic complexity. If the source code is unavailable, explain which white-box measurements cannot be established and use externally observable behavior instead.

e) Test adequacy and defect reporting

Prepare a traceability matrix linking transaction rules to test cases. Report actual outcomes, HTTP status codes, defects, severity, and reproduction steps. Evaluate the adequacy of the test suite using requirement coverage, decision-table coverage, and applicable code coverage when source code is available.

Expected deliverables: API inventory, Postman collection, request-response screenshots, decision table, state diagram, test-case sheet, defect report, and test adequacy summary.

Q4. Reverse Engineering a Commercial Off-The-Shelf (COTS) Application

Task Title: Functional, Compatibility, and Performance Testing of an Existing COTS Application

Tools: Apache JMeter, Selenium WebDriver, browser DevTools, spreadsheet software

A university procures a third-party Learning Management System (LMS) instead of developing its own. The university does not have access to the application's source code. The LMS supports login, course enrollment, assignment uploads, and result viewing.

The university wants to determine whether the purchased application meets its operational requirements before deployment.

Practical tasks

a) Reverse engineering of observable functionality

1. Explore the LMS interface and identify its main functions, navigation paths, input fields, and outputs.
2. Prepare a functional decomposition diagram of the LMS.
3. Derive testable requirements from approved specifications and observed behavior.
4. Identify assumptions, limitations, and behaviors requiring clarification from the vendor.

b) Black-box test design

1. Design equivalence classes and boundary tests for file size, file type, login fields, and enrollment inputs.
2. Use decision tables to test assignment submission rules.
3. Apply state transition testing to submission states such as Draft, Submitted, Late, and Rejected where supported.
4. Use pairwise testing for supported browser, operating system, and device combinations.
5. Apply exploratory testing to identify usability and navigation defects.

c) Tool-based functional and performance testing

1. Automate selected LMS workflows using Selenium.
2. Use JMeter against an authorized test environment to simulate a defined workload on supported HTTP endpoints.
3. Measure response times, throughput, and error rates.
4. Compare the observed results against predefined acceptance criteria.
5. Capture screenshots and export test reports.

d) COTS testing limitations

Explain why the lack of source-code access limits CFG construction, internal path coverage, and direct JaCoCo-style instrumentation. Identify alternative evidence, including requirements-based testing, API testing, interface testing, vendor documentation, and acceptance testing.

e) Test adequacy evaluation

Develop a weighted test coverage matrix for functionality, compatibility, security, performance, and usability. Identify untested risks and recommend whether the evidence supports acceptance, conditional acceptance, or rejection against the defined criteria.
Expected deliverables: Functional decomposition diagram, compatibility matrix, Selenium scripts, JMeter results, test-case spreadsheet, defect log, screenshots, and COTS acceptance recommendation.

Q5. Reverse Engineering a Python Application Using Pytest and Coverage.py

Task Title: Reverse Engineering, Path Testing, and Test Adequacy Evaluation of a Result Validation Program

Tools: Python, pytest, Coverage.py, Pylint or Ruff

A department receives an existing Python program that validates assessment marks and calculates pass/fail status. The original requirements and test suite are unavailable. The department suspects incorrect behavior for invalid inputs, boundary values, and combinations of marks.

Students must reconstruct the program's logic, design a comprehensive test suite, and use code-analysis tools to assess its quality.

Sample program

```
def validate_result(mark, attendance):
    if mark < 0 or mark > 100:
        return "Invalid Mark"

    if attendance < 0 or attendance > 100:
        return "Invalid Attendance"

    if attendance < 75:
        return "Attendance Shortage"

    if mark >= 50:
        return "Pass"

     return "Fail"
```

Practical tasks

a) Reverse engineering and specification recovery
1. Analyze the function's inputs, conditions, outputs, and decision order.
2. Prepare a flowchart and CFG.
3. Derive the functional rules implied by the program.
4. Identify missing requirements, such as whether exactly 75% attendance qualifies and how invalid data should be reported.
5. Separate confirmed behavior from requirements that need stakeholder approval.

b) Black-box test design
1. Apply Equivalence Class Partitioning to mark and attendance inputs.
2. Apply Boundary Value Analysis at 0, 49, 50, 74, 75, 100, and relevant out-of-range values.
3. Create a decision table covering valid marks, valid attendance, attendance eligibility, and pass/fail outcomes.
4. Use random testing to generate additional input pairs.
5. Use error guessing to investigate negative values, values exceeding 100, missing inputs, and incorrect data types.

c) White-box test design and automation

1. Identify the decision nodes and independent paths.
2. Calculate cyclomatic complexity from the CFG.
3. Write pytest tests for the major decision outcomes and boundary conditions.
4. Generate a Coverage.py report and identify uncovered statements and branches where branch measurement is enabled.
5. Add test cases to exercise the missing branches.

d) Additional white-box testing

1. Identify definitions and uses of mark and attendance.
2. Explain how data-flow testing can help identify incorrect or missing variable checks.
3. Propose mutations such as changing attendance < 75 to attendance <= 75, or mark >= 50 to mark > 50.
4. Explain which test cases should fail if the test suite effectively detects each mutation.

e) Evaluate test adequacy

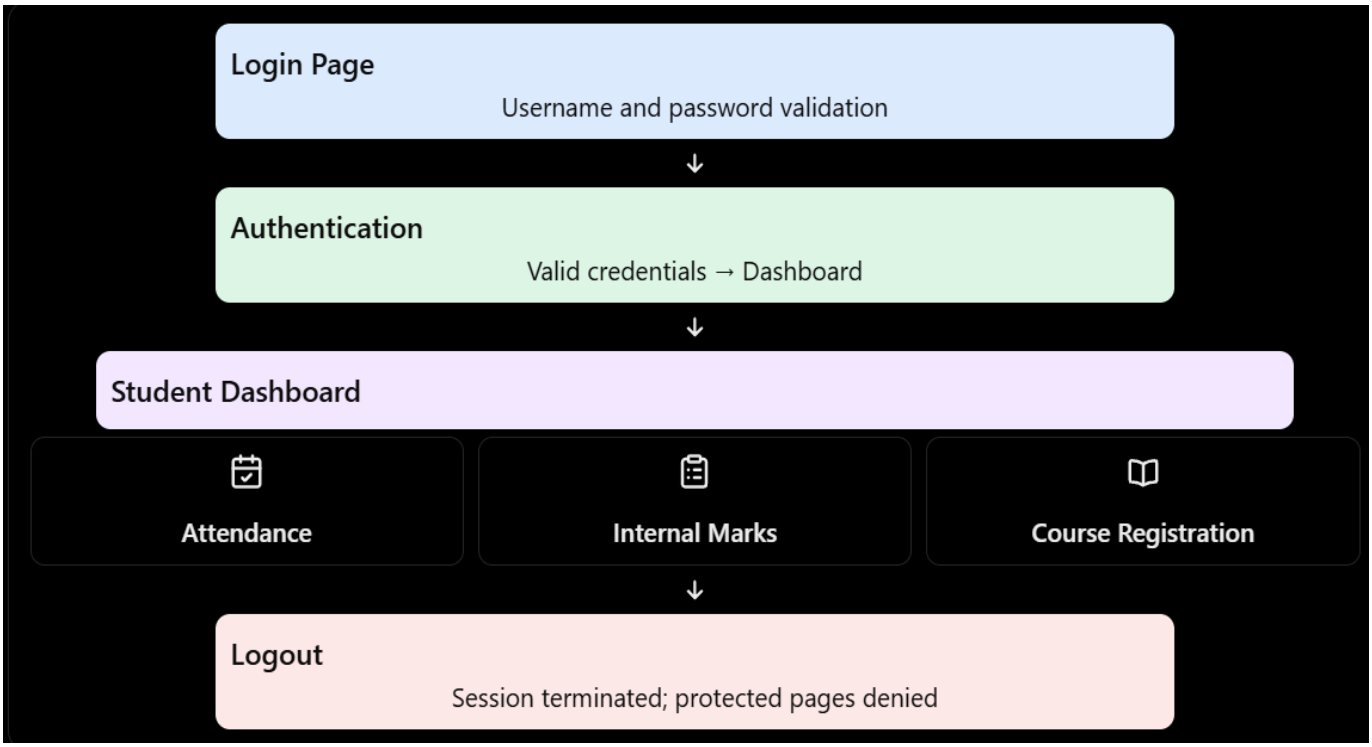
Compare the initial and improved test suites using statement coverage, branch coverage, boundary coverage, and requirements traceability. Explain why a high coverage percentage alone cannot prove that the program is correct.

Expected deliverables: Reverse-engineered specification, flowchart, CFG, complexity calculation, pytest scripts, Coverage.py reports, mutation-test analysis, defect log, and final adequacy evaluation.

ANSWERS

Q1. Reverse Engineering of a Black-Box Web Application Using Selenium
a) Application reconnaissance and reverse engineering
Aim: To study an existing student portal and identify its functionality without accessing its source code.

Functional model



Reconnaissance steps

1. Open the portal and inspect available pages, buttons, forms, links, and navigation.
2. Use Chrome DevTools → Elements to inspect HTML tags and attributes.
3. Use Network to examine requests, responses, status codes, and cookies.
4. Use Console to identify JavaScript errors.
5. Record observed behavior and compare it with expected behavior.

Preliminary functional requirements

- FR1: The portal shall accept a username and password.
- FR2: Valid credentials shall allow login.
- FR3: Invalid credentials shall display an error message.
- FR4: Successful login shall create an authenticated session.
- FR5: Authenticated users shall access authorized student pages.
- FR6: Logout shall terminate the session.
- FR7: Unauthenticated users shall not access protected pages.

Observation vs assumption: A displayed error message is an observation. Assuming the system uses a particular database or encryption algorithm is an assumption unless independently verified.

b) Black-box test design

Equivalence classes

Input	Valid class	Invalid class
Username	Registered, correctly formatted username	Empty, unknown, malformed username
Password	Correct password	Empty, incorrect password
Session	Active authenticated session	Missing, expired, or terminated session

Boundary Value Analysis should be applied to documented username and password length restrictions. Do not invent length limits if none are documented.

Sample test-case table

ID	Input / Action	Expected result
TC01	Valid username and password	Dashboard displayed
TC02	Incorrect password	Login rejected with error
TC03	Unknown username	Login rejected
TC04	Empty username	Validation message
TC05	Empty password	Validation message
TC06	Both fields empty	Required-field validation
TC07	Logout after login	Session terminated
TC08	Open protected page without login	Redirect or access denied
TC09	Use browser Back after logout	Protected content not accessible
TC10	Session timeout	Reauthentication required
TC11	Random username/password combinations	No crash or unauthorized access

For each test, record preconditions, input, steps, expected result, actual result, status, and defect ID. Actual results and defects must be filled in after execution.

c) Selenium automation

```
  import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException

URL = "http://localhost:8000/login"

class LoginTest(unittest.TestCase):
    def setUp(self):
        self.driver = webdriver.Chrome()
        self.wait = WebDriverWait(self.driver, 10)
        self.driver.get(URL)

    def test_empty_fields(self):
        self.driver.find_element(
            By.ID, "login-button"
        ).click()

        self.assertTrue(
            self.driver.find_element(
                By.ID, "error-message"
            ).is_displayed()
        )

    def test_invalid_login(self):
        self.driver.find_element(
            By.ID, "username"
        ).send_keys("wrong_user")
```

```
        self.driver.find_element(
            By.ID, "password"
        ).send_keys("wrong_password")

        self.driver.find_element(
            By.ID, "login-button"
        ).click()

        self.wait.until(
            EC.visibility_of_element_located(
                (By.ID, "error-message")
            )
        )

    def test_valid_login(self):
        self.driver.find_element(
            By.ID, "username"
        ).send_keys("test_student")

        self.driver.find_element(
            By.ID, "password"
        ).send_keys("TestPassword123")

        self.driver.find_element(
            By.ID, "login-button"
        ).click()

        self.wait.until(
            EC.url_contains("dashboard")
        )

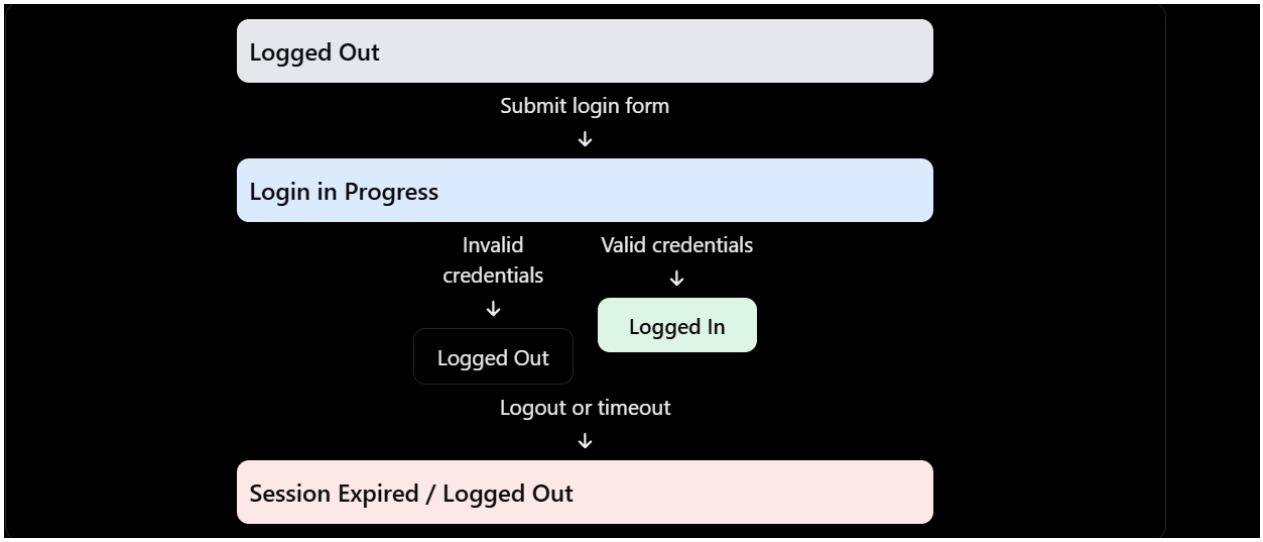
    def test_unauthorized_access(self):
        self.driver.get(
            "http://localhost:8000/attendance"
        )

        self.assertTrue(
            "login" in self.driver.current_url.lower()
            or "unauthorized" in self.driver.current_url.lower()
        )

    def tearDown(self):
        self.driver.quit()

if __name__ == "__main__":
    unittest.main()
```

d) State transition diagram



e) Test adequacy evaluation

$$\text{Execution Coverage} = \frac{\text{Executed Test Cases}}{\text{Planned Test Cases}} \times 100$$

$$\text{Pass Rate} = \frac{\text{Passed Test Cases}}{\text{Executed Test Cases}} \times 100$$

Example: If 8 of 10 planned tests are executed and 7 pass:

- Execution coverage = \(8/10 \times 100 = 80\%\)
- Pass rate = \(7/8 \times 100 = 87.5\%\)

Conclusion: Release readiness cannot be established from these percentages alone. Session security, unauthorized access, browser compatibility, and all critical requirements must also be evaluated.

Q2. Reverse Engineering a Java Application Using JaCoCo

a) Logic reconstruction

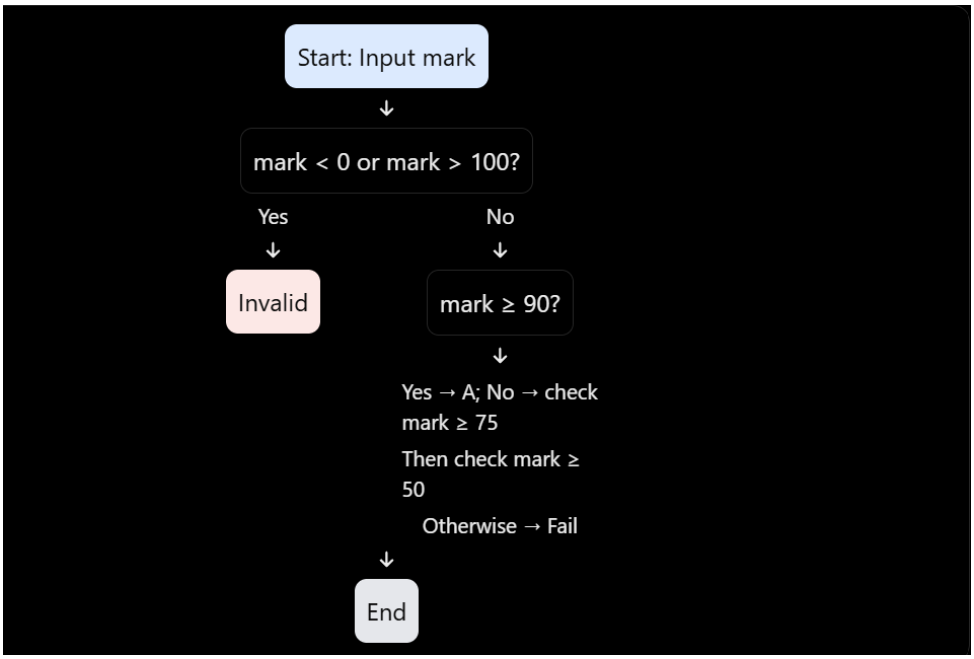
Aim: To understand the grading program, create tests, and measure code coverage.

Implied grading rules

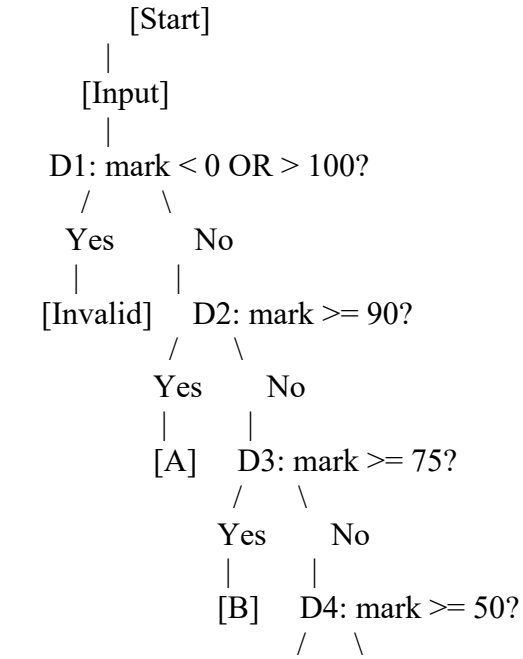
Mark condition	Output
Mark below 0 or above 100	Invalid
90–100	A
75–89	B
50–74	C
0–49	Fail

The input type is int. Handling null values or non-integer inputs requires additional input-parsing logic outside this method.

Flowchart



Control Flow Graph (CFG)



Yes No
| |
[C] [Fail]

b) Black-box test cases

Test input	Expected output
-1	Invalid
0	Fail
49	Fail
50	C
74	C
75	B
89	B
90	A
100	A
101	Invalid

These cases cover the lower and upper invalid boundaries, valid range endpoints, and all grade thresholds.

c) White-box testing

There are four decision statements:

1. Invalid-range condition
2. mark >= 90
3. mark >= 75
4. mark >= 50

For a single connected control-flow graph:

V(G)=E-N+2P

where E = edges,N = nodes, and P= for one connected component.

Alternatively, for this structured program:

V(G)= {Number of decisions}+1=4+1=\boxed{5}

There are five independent paths corresponding to Invalid, A, B, C, and Fail.

JUnit test script

Save as ResultCalculatorTest.java in a Maven project's test source folder.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class ResultCalculatorTest {

    @Test
    void invalidMarks() {
        assertEquals("Invalid",
            ResultCalculator.calculateGrade(-1));
        assertEquals("Invalid",
            ResultCalculator.calculateGrade(101));
    }

    @Test
    void failBoundary() {
        assertEquals("Fail",
            ResultCalculator.calculateGrade(0));
        assertEquals("Fail",
            ResultCalculator.calculateGrade(49));
    }

    @Test
    void gradeCBoundary() {
        assertEquals("C",
            ResultCalculator.calculateGrade(50));
```

```
        assertEquals("C",
            ResultCalculator.calculateGrade(74));
    }

    @Test
    void gradeBBoundary() {
        assertEquals("B",
            ResultCalculator.calculateGrade(75));
        assertEquals("B",
            ResultCalculator.calculateGrade(89));
    }

    @Test
    void gradeABoundary() {
        assertEquals("A",
            ResultCalculator.calculateGrade(90));
        assertEquals("A",
            ResultCalculator.calculateGrade(100));
    }
}
```

Maven and JaCoCo configuration

Add the following plugins inside the build section of pom.xml. Include a compatible JUnit Jupiter dependency in the dependencies section.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.14</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Run:

```
mvn clean test
```

Open the generated report:

```
target/site/jacoco/index.html
```

Coverage comparison

Metric	Meaning
Instruction coverage	Executed bytecode instructions
Line coverage	Source lines executed
Branch coverage	Decision outcomes exercised
Method coverage	Methods invoked by tests
Complexity	Complexity of control flow and untested complexity

Statement coverage does not guarantee branch coverage because a test can execute a decision statement while exercising only one of its outcomes.

d) Test adequacy evaluation

The ten test inputs exercise all five principal outcomes and both outcomes of the four decisions when considered together. For this sample, they are designed to achieve full line and branch coverage of calculateGrade.

However, full coverage does not prove correctness for every requirement. The grading thresholds must be confirmed with the university, and additional checks may be needed for input parsing, integer conversion, and integration with student records.

Conclusion: JaCoCo helps identify unexecuted code and branches. The improved test suite gives stronger evidence, but correctness must also be assessed against approved grading requirements.

Q3. Reverse Engineering and API Testing of a Banking Application

a) API reverse engineering

Aim: To reconstruct the observable behavior of a banking REST API and verify transaction rules using Postman.

Use a mock or locally hosted application with test accounts only.

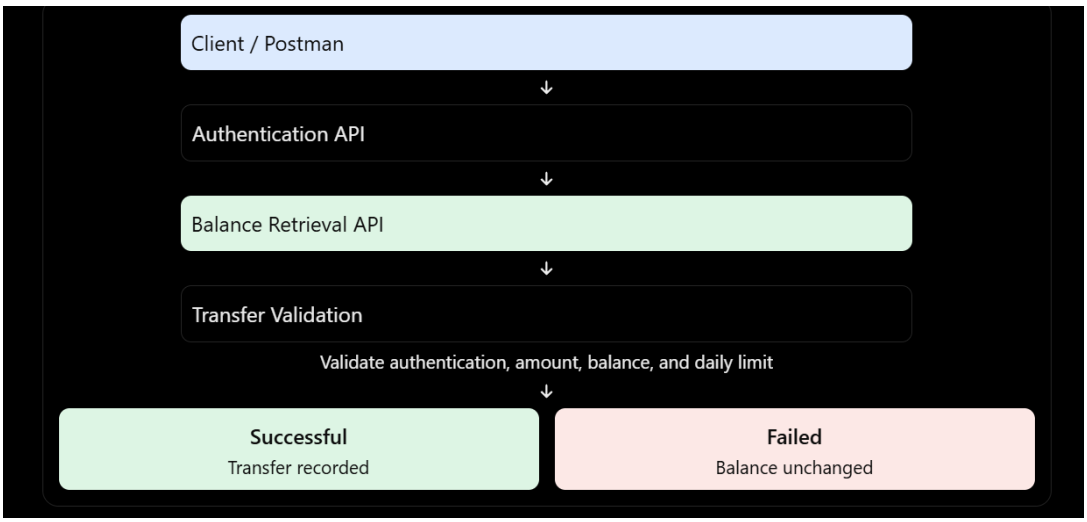
API inventory

The following is an illustrative API design; verify actual endpoints and status codes against the supplied application.

Method	Endpoint	Purpose
POST	/api/login	Authenticate user
GET	/api/accounts/{id}/balance	Retrieve account balance
POST	/api/transfers	Initiate transfer
GET	/api/transfers/{id}	Retrieve transaction status

Record the request body, response body, HTTP status code, authentication headers, and observed side effects for every test.

API interaction diagram



b) Black-box test design

Equivalence classes

- Valid amount: Greater than zero, within available balance, and within the daily limit.
- Invalid amount: Zero, negative, malformed, or non-numeric.
- Insufficient balance: Amount exceeds available balance.
- Daily limit exceeded: Amount exceeds the remaining permitted daily limit.
- Invalid authentication: Missing, expired, or incorrect credentials.
- Invalid account: Unknown or incorrectly formatted account identifier.

Sample Postman test cases

Assume a mock account has a balance of ₹10,000 and a remaining daily limit of ₹5,000. These are test assumptions, not actual banking data.

ID	Test input	Expected result
API01	Amount ₹1	Accept if all other rules pass
API02	Amount ₹0	Reject
API03	Amount −₹1	Reject
API04	Amount ₹10,001	Reject: insufficient balance
API05	Amount ₹5,001	Reject: daily limit exceeded
API06	Amount ₹5,000	Accept if the limit is inclusive and other rules pass
API07	No authentication token	Reject; no transfer
API08	Missing amount field	Reject validation error
API09	Amount supplied as text	Reject invalid data type
API10	Invalid account ID	Reject or return not found

API11	Unsupported HTTP method	Return appropriate method error
API12	Repeat the same transfer request	No duplicate transfer if idempotency is required

The exact response codes must be verified. Common conventions include 400 for malformed input, 401 for unauthenticated requests, 403 for forbidden actions, 404 for unknown resources, and 405 for unsupported methods. A business-rule failure may use 400 or another documented response.

Decision table

Authenticated?	Sufficient balance?	Within daily limit?	Expected result
No	Yes	Yes	Reject
Yes	No	Yes	Reject
Yes	Yes	No	Reject
Yes	No	No	Reject
Yes	Yes	Yes	Accept

c) State transition testing



Test that valid transfers reach the appropriate final state. Rejected transfers must not reduce the balance. Test repeated requests, timeouts, retries, and cancellation where supported. Verify duplicate prevention using an idempotency key if the API supports one.

d) White-box analysis

If source code is supplied, construct the control-flow graph, count decision nodes, and calculate cyclomatic complexity.

If the source code is unavailable, internal path coverage and code-level cyclomatic complexity cannot be measured reliably. Use black-box evidence, API contract testing, response analysis, and balance reconciliation instead.

e) Defect report and adequacy

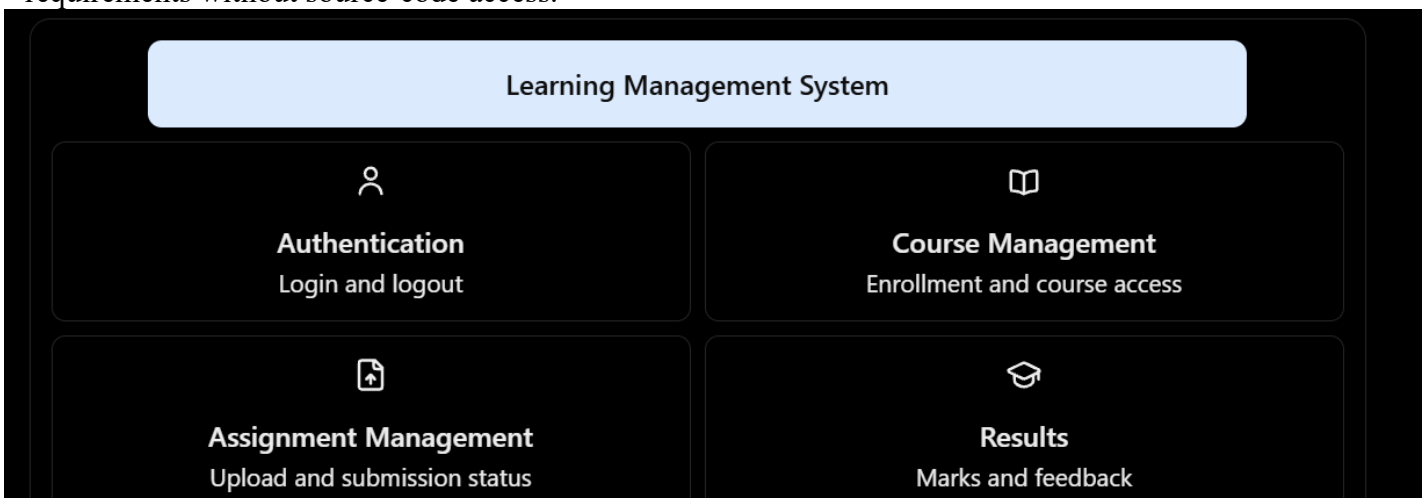
Defect ID	Example defect	Severity
BANK-01	Unauthenticated transfer succeeds	Critical
BANK-02	Failed transfer reduces balance	Critical
BANK-03	Daily limit is not enforced	High
BANK-04	Repeated request creates duplicate transfer	High
BANK-05	Malformed amount causes server error	Medium

Conclusion: API testing is adequate only when all specified business rules, important invalid inputs, authentication controls, transaction states, and financial side effects have been checked. Code coverage is applicable only when source code is available.

Q4. Reverse Engineering a Commercial Off-The-Shelf (COTS) Application

a) Functional decomposition

Aim: To evaluate whether a purchased Learning Management System (LMS) satisfies the university's requirements without source-code access.



b) Black-box test design

Equivalence classes and boundary tests

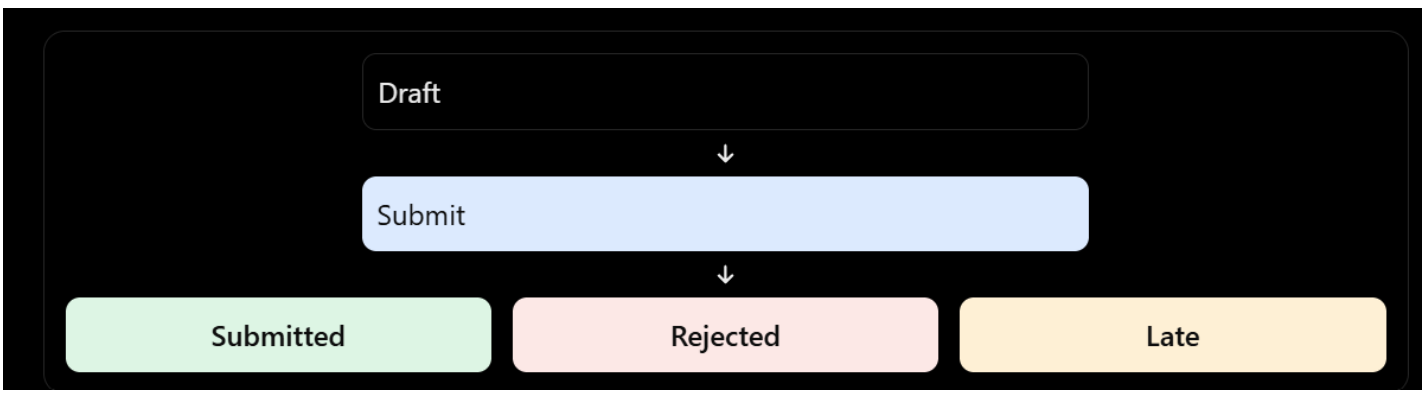
Feature	Valid case	Invalid / boundary case
Login	Correct credentials	Empty fields, incorrect password
File size	Within documented limit	Zero-byte file, limit, limit + 1 byte
File type	Approved extension and content	Unsupported extension or mismatched content
Enrollment	Eligible student and open course	Ineligible student, closed course
Assignment deadline	Before deadline	Exactly at deadline, after deadline

Use actual documented file-size limits and accepted file types. Test that the server validates uploads, rather than relying only on the browser's file picker.

Assignment decision table

Valid file?	Before deadline?	User eligible?	Expected result
Yes	Yes	Yes	Submission accepted
No	Yes	Yes	Rejected
Yes	No	Yes	Late submission or rejection, according to policy
Yes	Yes	No	Access denied
No	No	No	Rejected

Submission state diagram



Compatibility testing

Use pairwise testing to cover combinations of supported browsers, operating systems, and devices. For example, test Chrome on Windows, Edge on Windows, Chrome on Android, and Safari on iOS when those platforms are supported.

c) Selenium and JMeter testing

Use Selenium for user workflows:

- Login and logout.

- Course enrollment.
- Assignment upload.
- Result viewing.
- Navigation and validation.

Use JMeter against authorized test endpoints to simulate a predefined workload. Record:

- Average and percentile response times.
- Throughput (requests per second).
- Error rate.
- Behavior under expected and peak load.

Acceptance criteria must be defined before execution. For example, an illustrative target could be a 95th-percentile response time below 2 seconds at the agreed workload. This is not a universal LMS requirement.

d) COTS testing limitations

Without source code, testers generally cannot construct a reliable internal control-flow graph, measure internal path coverage, or directly instrument the application with JaCoCo.

Alternative approaches include:

1. Requirements-based functional testing.
2. API and interface testing.
3. Browser compatibility testing.
4. Security and access-control testing.
5. Performance testing using JMeter.
6. Vendor documentation review and user acceptance testing.

e) Weighted test coverage and acceptance

An illustrative weighted matrix is:

Category	Weight	Coverage
Functionality	35%	90%
Compatibility	20%	80%
Security	20%	70%
Performance	15%	85%
Usability	10%	90%

Recommendation: Conditional acceptance may be appropriate if the remaining gaps are non-critical and an agreed remediation plan exists. A critical security failure or failure of a mandatory requirement should block acceptance regardless of the weighted score.

Q5. Reverse Engineering a Python Application Using Pytest and Coverage.py

a) Reverse engineering and specification recovery

Aim: To reconstruct the validation logic, design test cases, and measure statement and branch coverage.

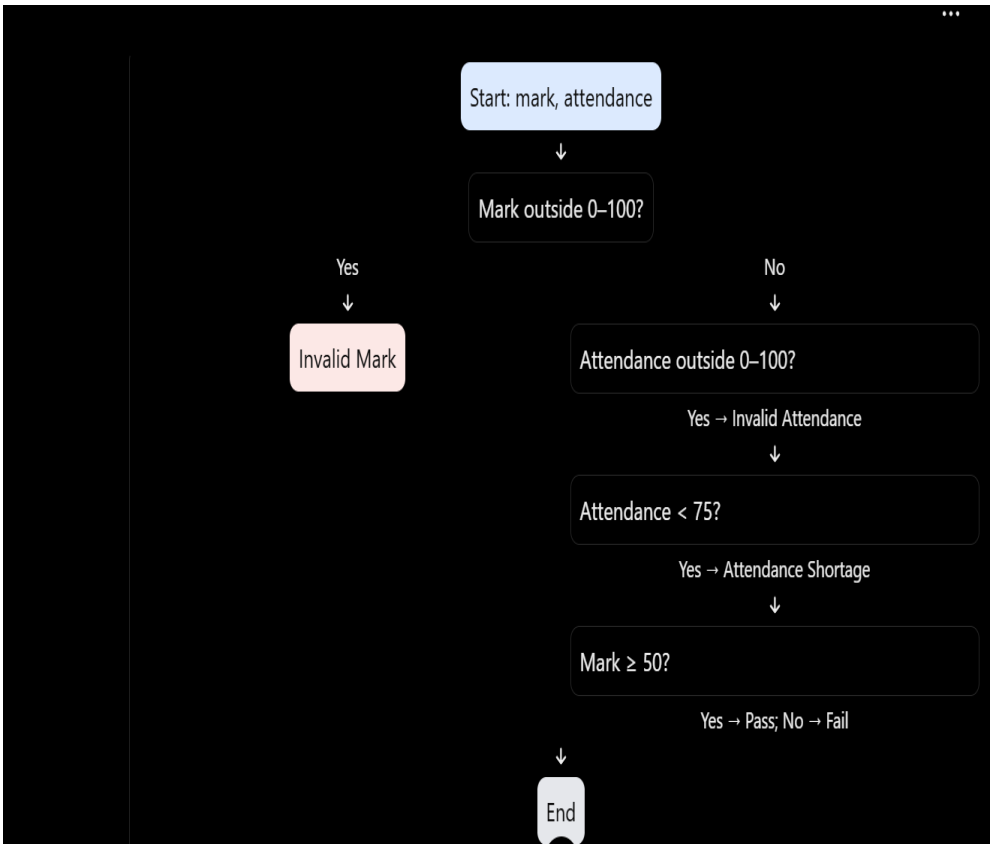
Reconstructed requirements

The given function accepts two inputs: mark and attendance.

1. If the mark is outside 0–100, return "Invalid Mark".
2. If attendance is outside 0–100, return "Invalid Attendance".
3. If attendance is below 75%, return "Attendance Shortage".
4. If attendance is at least 75% and mark is at least 50, return "Pass".
5. Otherwise, return "Fail".

The order of validation matters. If both inputs are invalid, the function returns "Invalid Mark" first.

Flowchart



b) Black-box test design

Equivalence classes

Input	Equivalence classes
Mark	Below 0, 0–100, above 100
Attendance	Below 0, 0–100, above 100
Attendance eligibility	Below 75, exactly 75, above 75
Result	Pass, Fail, Attendance Shortage, Invalid Mark, Invalid Attendance

Decision table

Mark valid?	Attendance valid?	Attendance ≥ 75?	Mark ≥ 50?	Expected output
No	Any	Any	Any	Invalid Mark
Yes	No	Any	Any	Invalid Attendance
Yes	Yes	No	Any	Attendance Shortage
Yes	Yes	Yes	Yes	Pass
Yes	Yes	Yes	No	Fail

Sample test cases

ID	Mark	Attendance	Expected output
PY01	-1	80	Invalid Mark
PY02	101	80	Invalid Mark
PY03	50	101	Invalid Attendance

PY04	50	-1	Invalid Attendance
PY05	80	74	Attendance Shortage
PY06	80	75	Pass
PY07	49	75	Fail
PY08	50	100	Pass
PY09	0	75	Fail
PY10	100	100	Pass

Additional tests should include random input pairs, floating-point values, strings, None, and missing function arguments. The current function does not explicitly validate data types and may raise exceptions for unsupported inputs.

c) White-box testing and cyclomatic complexity

There are four decision statements:

1. Mark validation.
2. Attendance validation.
3. Attendance eligibility.
4. Pass/fail condition.

Therefore,

V(G)=\text{Number of decisions}+1

V(G)=4+1=\boxed{5}

The program has five principal independent paths:

- Invalid mark.
- Valid mark but invalid attendance.
- Valid inputs with attendance shortage.
- Eligible attendance with a passing mark.
- Eligible attendance with a failing mark.

Pytest test script

Save the original function in result.py and the following tests in test_result.py.

from result import validate_resultdef test_invalid_mark(): assert validate_result(-1, 80) == "Invalid Mark" assert validate_result(101, 80) == "Invalid Mark"def test_invalid_attendance(): assert validate_result(50, -1) == "Invalid Attendance" assert validate_result(50, 101) == "Invalid Attendance"def test_attendance_shortage(): assert validate_result(80, 74) == "Attendance Shortage"def test_pass_boundary(): assert validate_result(50, 75) == "Pass"def test_fail_boundary(): assert validate_result(49, 75) == "Fail"def test_full_attendance(): assert validate_result(100, 100) == "Pass"

Run the tests:

pip install pytest coverage

pytest -v

d) Coverage.py report

Run the tests with branch coverage enabled:

coverage run --branch -m pytest

coverage report -m

coverage html

Open the generated report:

htmlcov/index.html

Coverage.py reports executed statements and, with branch measurement enabled, observed branch outcomes.

Statement vs branch coverage

Coverage type	Meaning
Statement coverage	Measures whether executable statements ran
Branch coverage	Measures whether each possible branch was taken
Boundary coverage	Measures whether important boundary values were tested
Requirements coverage	Measures whether specified requirements have corresponding tests

Executing every statement does not necessarily exercise every branch. For example, a test may execute the pass/fail statement but never test the failing-mark outcome.

The sample tests are designed to cover the major outcomes and both sides of the four decisions. Actual coverage percentages must be taken from the generated report rather than assumed.

Data-flow testing

Data-flow testing tracks where variables are defined and used.

- mark is checked for validity and later used in the pass/fail decision.
- attendance is checked for validity and later used in the eligibility decision.

It can reveal missing validation, incorrect variable use, or checks performed in the wrong order.

Mutation testing

Mutation testing deliberately introduces small code changes to check whether the test suite detects them.

Mutation	**Test that should detect it**	**Reason**
Change attendance < 75 to attendance <= 75	Mark 80, attendance 75	Expected Pass; mutant returns Attendance Shortage
Change mark >= 50 to mark > 50	Mark 50, attendance 75	Expected Pass; mutant returns Fail

A test suite that fails when these mutations are introduced provides evidence that it detects the corresponding boundary errors.

e) Test adequacy evaluation

Compare the initial and improved suites using the following measures:

Metric	**Initial suite**	**Improved suite**
Statement coverage	Measure using Coverage.py	Measure using Coverage.py
Branch coverage	Measure using Coverage.py	Measure using Coverage.py
Boundary coverage	May be incomplete	Includes 49/50 and 74/75
Requirements traceability	May be incomplete	Each principal rule linked to tests
Mutation detection	Not established	Verify by running mutations

Conclusion: The improved test suite provides stronger evidence because it covers the principal execution paths, validation rules, and important boundaries. High code coverage alone cannot prove correctness; the test results must also match the approved requirements, and invalid data types, missing inputs, and untested combinations must be considered.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation